

Analisi variabili progetto (vocabolario progetto)

Riassunto requisiti

Requisiti funzionali rigidi:

- tracking ambientale: registrazione e pubblicazione periodica di parametri fisici (temperatura, umidità, vibrazioni, ecc.);
- firma digitale alla sorgente (blindatura dei dati): i parametri ambientali rilevati dai sensori devono essere firmati digitalmente alla sorgente tramite chiave privata del dispositivo prima della trasmissione. Il sistema non deve consentire ad alcun attore umano (operatore di magazzino, autista o manager) di modificare o cancellare i dati storici memorizzati, pena l'invalidazione matematica della firma crittografica rilevata lato Cloud;
- gestione dispositivi (devices): il sistema deve consentire di:
 1. censire (provision) e dismettere (decommission) i dispositivi IoT;
 2. elencarli (list);
 3. associarli o dissociarli liberamente a specifiche spedizioni;
 4. configurarli (definire la frequenza di campionamento, la frequenza di upload e quali metriche raccogliere).
- gestione spedizioni (shipments): gli attori abilitati devono poter creare, modificare, ispezionare e cancellare le spedizioni. Ogni spedizione deve poter essere avviata, messa in pausa, ripresa o arrestata (start/stop/pause/resume);
- dashboard e allarmi: la dashboard cloud deve evidenziare graficamente le condizioni di allarme "out of range" (parametri fuori norma) e rendere i dati accessibili in modo continuo a 4 categorie di utenti: manager, autisti, consumatori e autorità di certificazione.

Requisiti non funzionali rigidi (vincoli tecnologici e di performance):

- dispositivi low-Cost e riusabili: i sensori IoT devono essere economici e riutilizzabili per spedizioni e merci differenti;
- alimentazione e connettività dei sensori: i dispositivi IoT funzionano esclusivamente a batteria e non hanno accesso diretto ad Internet (escluso il Wifi, escluso il 5G/4G per il sensore fisico). Devono comunicare tramite metodi a basso consumo energetico come il Bluetooth Low Energy (BLE);
- monitoraggio batteria: Il sistema deve monitorare lo stato di carica della batteria del dispositivo IoT e inviare notifiche quando il livello è basso (manutenzione, es. <15%);
- architettura SaaS cloud: la piattaforma deve poggiare su un'infrastruttura Cloud erogata come servizio (SaaS) per la gestione e la visualizzazione dei dati;
- interoperabilità via API: esposizione di interfacce web (API) per consentire lo scambio sicuro e trasparente dei dati con applicativi ed enti terzi (come gli enti di certificazione di qualità/sostenibilità).

Requisiti conseguenti alle variabili (aspettative, dominio, hp di dominio): ---da rivedere

- sincronizzazione automatica: ci si aspetta che l'app Gateway (sia sul telefono del magazziniere che su quello dell'autista) tenti automaticamente la connessione BLE e scarichi i dati accumulati nella memoria del sensore non appena questo entra nel raggio di copertura radio (inferiore a 10 metri);
- ultima possibilità: in caso di guasto ad un box/sensore di trasporto, mantenere nel veicolo uno o più box/sensori di emergenza (vedi 'Aspettative e assunzioni').

---- Probabilmente queste possono essere mitigazioni a ostacoli più che requisiti originali!

Software to-be

Seguendo i requisiti appena descritti, il software-to-be è composto da quattro macro-sistemi software che costituiscono l'intera macchina:

1. applicazione mobile gateway (app gateway autista):
 - ruolo: gira sullo smartphone o tablet del conducente;
 - responsabilità: fa da tramite logico (ripetitore) tra i sensori fisici e il cloud. Riceve i pacchetti telemetrici firmati digitalmente via Bluetooth Low Energy (BLE) e li inoltra via connessione cellulare (4G/5G) al backend cloud SaaS;
2. backend cloud SaaS (infrastruttura centrale):
 - ruolo: è l'ossatura logica del server centrale;
 - responsabilità: riceve i dati dall'app gateway, si occupa di verificare la firma crittografica asimmetrica per garantire che il dato non sia stato manomesso lungo la tratta, calcola se i dati violano le soglie (generazione degli allarmi) e memorizza storicamente i viaggi nel database sicuro;
3. dashboard web (interfaccia utente):
 - ruolo: l'applicazione web centrale accessibile tramite browser;
 - responsabilità: rende i dati accessibili ai manager (per creare spedizioni, censire sensori fisici), autisti (per monitorare l'andamento live), consumatori (tramite scansione QR Code) e autorità di certificazione;
4. API web pubbliche (API gateway interoperabile):
 - ruolo: endpoint sicuri (REST/JSON) esposti verso l'esterno;
 - responsabilità: consentono lo scambio programmatico e l'audit dei dati con sistemi terzi, come i sistemi informatici gestionali (ERP) delle aziende clienti o i server degli enti di certificazione della qualità di filiera.

L'Ambiente

Agenti (Who - ruoli attivi)

Gli agenti dell'ambiente sono entità autonome dotate di comportamento proprio. Gli obiettivi che assegniamo a loro nel Goal Diagram non sono requisiti software, ma si chiamano aspettative (expectations), perché non possiamo forzarli via codice ma dobbiamo assumere che si comportino bene.

- Agenti umani (user roles):
 - manager logistico (SaaS administrator): l'operatore dell'azienda logistica che supervisiona l'intera flotta e la catena di distribuzione. Controlla l'associazione logica dei dispositivi ai lotti dei prodotti e monitora lo stato globale delle spedizioni e gli allarmi tramite dashboard web desktop;
 - autista (conducente del mezzo): l'operatore fisico che conduce il veicolo di trasporto (camion refrigerato) lungo la tratta logistica. Monitora le notifiche di allarme acustico/visivo sul Gateway mobile (smartphone) in cabina e controlla fisicamente lo stato del viaggio (avvio, sospensione, termine);
 - operatore di magazzino: l'operatore addetto allo stoccaggio fisico e alle procedure di carico/scarico merci presso gli hub logistici. Controlla fisicamente il posizionamento del sensore all'interno dei colli/scatole refrigerate e l'accensione dell'hardware;
 - consumatore finale: il destinatario finale del prodotto (farmacia, ospedale, galleria d'arte, o singolo cittadino) che riceve o vuole acquistare la merce. Monitora/visualizza l'integrità storica del processo termico scansionando il QR Code stampato sulla scatola tramite browser web mobile;
 - produttore dei dispositivi: l'azienda esterna che produce e calibra l'hardware (i sensori IoT). Controlla la generazione della chiave privata/pubblica iniziale del sensore e la calibrazione fisica dei trasduttori termici.
- Agenti hardware:
 - sensore IoT (dispositivo hardware di telemetria): il modulo fisico (microcontrollore con batteria, trasduttori termici/umidità e antenna BLE) posizionato all'interno della scatola dei prodotti. Fa parte dell'ambiente perché gira su un firmware ultra-low-level pre-installato e non modificabile dal nostro sistema Cloud/App Gateway, se non per la frequenza di campionamento e le grandezze da misurare. Controlla il campionamento dei parametri fisici reali, la scrittura delle letture nella sua flash memory locale, la firma crittografica asimmetrica del payload alla sorgente e la trasmissione dei dati via BLE. Monitora lo stato fisico della propria batteria e i timer interni di campionamento.

Entità fisiche passive (What - mondo fisico)

Queste sono le entità del mondo reale che non prendono decisioni autonome (non sono agenti), ma possiedono proprietà fisiche e stati dinamici che il sistema deve monitorare e mappare in classi concettuali.

- prodotto/merce: ogni prodotto, a seconda della tipologia di appartenenza (farmaco, opera d'arte, piante, ...) possiede dei range di grandezze fisiche di riferimento (temperatura, umidità, vibrazioni, ...) e un tempo massimo di esposizione fuori soglia (es. 15 minuti) prima del deterioramento irreversibile;
- cella/box/package: volume d'aria interna, coefficiente di isolamento termico, stato della porta (Aperta/Chiusa), stato dell'alimentatore refrigerante o che controlla la grandezza in questione (acceso/spento/in anomalia);

- tratta stradale/viaggio: distanza chilometrica reale, presenza di ostacoli fisici (es. gallerie stradali che schermano la connettività GPS/4G), temperatura atmosferica esterna lungo la tratta.

Variabili (modello a 4 variabili)

Come risultato dell'ambiente, possiamo mappare le seguenti variabili:

1. variabili monitorate (M - stati reali del mondo):
 - a. grandezza fisica: la temperatura/umidità/vibrazione (ecc. ecc.) reale che circonda il prodotto all'interno del contenitore/box;
 - b. carica batteria: lo stato chimico/elettrico reale della carica della batteria al litio del sensore;
 - c. connessione BLE: la presenza di un canale radio Bluetooth Low Energy stabile (distanza fisica < 10 metri) tra il sensore IoT e il dispositivo Gateway circostante (es. smartphone dell'autista o tablet/terminale del magazziniere);
2. variabili controllate (C - stati del mondo che vogliamo imporre):
 - a. alert conducente: lo stato visivo e/o acustico dello schermo e del cicalino dello smartphone dell'autista (attivo/disattivo);
 - b. status spedizione: la marcatura formale dello stato del viaggio nel database centrale del cloud, accessibile a tutti gli attori. Può assumere i valori conforme, sospetta manomissione, parzialmente danneggiato, danneggiato irreversibilmente;
 - c. incidente: flag di diagnostica che rileva anomalie di sistema o incidenti fisici (T/F).
3. variabili di input (I - dati digitali che il software riceve):
 - a. valoreLetto (real): il valore numerico digitalizzato inviato dal sensore BLE (es. 4.5° nel pacchetto dati);
 - b. livelloBatteria (integer): il valore percentuale (0-100) letto dal firmware e trasmesso via BLE;
 - c. firmadigitale (string): la firma crittografica generata dal sensore IoT cifrando l'hash della lettura con la sua chiave privata;
 - d. pulsanteAvviaViaggio (boolean): input digitale generato dall'autista premendo il tasto "Start" sull'app/dashboard di controllo.
4. variabili di output (O - comandi digitali inviati dal software):
 - a. triggerNotificaPush (boolean): comando inviato dal gestore allarme del cloud all'App mobile dell'autista per forzare l'attivazione della suoneria di allarme;
 - b. codiceQRGenerato (string): stringa URL crittografata inviata dal cloud per la generazione del codice QR sulla dashboard del manager;
 - c. statoverificaFirma (boolean): flag logico generato dall'verificatore di integrità del cloud che indica se la firma del pacchetto dati è matematica e integra (T/F).

Proprietà di dominio (Dom)

Queste sono le leggi fisiche immutabili della natura, della termodinamica, della crittografia o della matematica che regolano il funzionamento del mondo reale a prescindere dal nostro software. Sono dichiarazioni descrittive, il software non può violarle o cambiarle.

- inerzia dello scambio termofisico: se i sistemi di condizionamento attivo o di isolamento passivo del contenitore logistico smettono di funzionare, le grandezze fisiche interne (temperatura, umidità, ...) non variano istantaneamente, ma si modificano gradualmente seguendo le leggi dello scambio termodinamico e di massa con l'ambiente esterno. Il concetto potrebbe seguire leggi diverse per vibrazioni o altre grandezze di natura radicalmente differente;
- soglie di tolleranza fisica: ciascuna tipologia di merce fragile possiede tolleranze fisiche specifiche per ogni parametro ambientale monitorato P (es. temperatura, umidità, accelerazione/urti, radiazione UV). Se il valore fisico reale supera la soglia di tolleranza massima consentita P_{max} o scende sotto la soglia mini P_{min} per un tempo cumulativo superiore a un intervallo critico, la merce subisce un deterioramento o danneggiamento irreversibile;
- battery death: se il livello di carica della batteria fisica di un dispositivo di campionamento scende allo 0%, esso si spegne istantaneamente, interrompendo qualsiasi operazione di misurazione telemetrica e di trasmissione radio BLE;
- inviolabilità del dato: data l'adozione di algoritmi crittografici asimmetrici standard (es. ECDSA) per la firma alla sorgente, l'alterazione dei dati fisici campionati o la falsificazione della firma senza la conoscenza della chiave privata del sensore IoT risulti computazionalmente impraticabile;
- schermatura fisica: la portata di trasmissione di un segnale radio ad alta frequenza come il Bluetooth Low Energy (BLE), se ostacolata da barriere metalliche (es. pareti schermate di un container o di una cella frigorifera), è limitata a una distanza fisica massima di circa 10 metri.

Ipotesi di dominio

Rappresentano stati di fatto, proprietà fisiche o vincoli tecnologici del mondo reale che assumiamo come costanti e veri. Sono assunzioni descrittive.

- copertura rete: si assume che lungo la tratta stradale del trasporto vi sia copertura di rete cellulare mobile (4G/5G) per almeno l'85% della durata temporale del viaggio;
- accuratezza dei sensori: assumiamo che per ogni grandezza fisica P monitorata (temperatura, umidità, vibrazioni, ecc.), l'errore di misurazione intrinseco dell'hardware di fabbrica sia limitato entro una tolleranza massima accettabile E_p definita nelle specifiche del produttore, garantendo che tale tolleranza non pregiudichi la rilevazione statistica di sfioramento delle soglie critiche della merce;
- integrità meccanica: assumiamo che i colli contenenti la merce e i sensori non vengano distrutti o danneggiati meccanicamente (es. schiacciamento del container) durante le normali operazioni di trasporto;
- capacità di buffering del sensore: si assume che il dispositivo sensore IoT sia dotato di memoria non volatile interna in grado di memorizzare storicamente le misurazioni

effettuate in assenza di una connessione BLE attiva con il Gateway, garantendo che nessun dato campionato vada perduto;

- alimentazione sensori: si assume che i dispositivi sensori IoT funzionino esclusivamente con batteria integrata e non abbiano un modulo di connettività diretta a Internet (es. SIM a bordo), ma comunichino unicamente tramite protocolli a corto raggio (BLE) con i Gateway circostanti.

Aspettative e assunzioni

Questi sono i comportamenti desiderati degli attori umani e dell'hardware (agenti) che non possiamo codificare noi nel software, ma che dobbiamo assumere come veri affinché il sistema globale funzioni e garantisca la sicurezza del trasporto. Sono dichiarazioni prescrittive.

- reazione immediata all'allarme: ci si aspetta che il conducente del veicolo logistico, non appena l'applicazione mobile Gateway emette un allarme acustico/visivo continuativo di grandezza fuori soglia, accosti in sicurezza entro 10 minuti per verificare l'alimentazione della cella o spostare i prodotti nei box d'emergenza;
- campionamento firmware: ci si aspetta che il dispositivo sensore IoT sia correttamente calibrato all'origine dal produttore, mantenga attiva l'esecuzione del proprio ciclo di campionamento e memorizzi in modo sicuro le letture nella memoria flash locale secondo la frequenza di campionamento configurata (es. valore di default pari a 60 secondi);
- associazione fisica corretta: ci si aspetta che l'operatore di magazzino posizioni il sensore all'interno dell'esatta scatola del prodotto ad esso associata logicamente a sistema, e non in un'altra, prima che il camion lasci l'hub di spedizione;
- vicinanza BLE: ci si aspetta che lo smartphone dell'autista (il nostro Gateway software) sia mantenuto costantemente acceso, con l'interfaccia Bluetooth attiva, e posizionato all'interno della cabina del veicolo di trasporto (a meno di 10 metri dal cassone refrigerato) per consentire la sincronizzazione BLE continua e stabile;
- associazione logica iniziale: ci si aspetta che il manager logistico esegua l'associazione tra sensore e spedizione correttamente tramite la dashboard cloud prima che il veicolo parta dall'hub;
- verifica iniziale del magazziniere: ci si aspetta che l'operatore di magazzino al hub di partenza, subito dopo aver acceso e posizionato fisicamente il sensore nella scatola, esegua una prima sincronizzazione BLE tramite l'app per verificare che il dispositivo stia campionando correttamente e registrare lo "stato zero" della spedizione sul cloud;
- scarico finale alla consegna: ci si aspetta che al momento della consegna della merce al cliente finale, il destinatario esegua un'ultima scansione/sincronizzazione BLE prima di dichiarare chiusa la spedizione, per garantire che tutte le misurazioni registrate fino all'ultimo secondo vengano caricate e certificate sul cloud;
- disattivazione per risparmio energetico (fine viaggio): ci si aspetta che l'operatore logistico dell'hub di arrivo, subito dopo aver completato la sincronizzazione finale dei dati e la dissociazione del sensore dal collo consegnato, provveda a spegnere il dispositivo (o a impostarlo in modalità deep-sleep a bassissimo consumo) per

arrestare immediatamente il campionamento attivo e preservare la carica della batteria per i viaggi successivi.

Punti aperti (x Mattia)

- nome sistema: bisogna dare un nome al sistema per togliere ambiguità nei testi quando ci riferiamo all'architettura scopo del progetto. Come titolo della repo git avevo messo ColdChain ma è errato dato che non c'entra solo con il freddo ma anche altre grandezze. Una roba tipo Fragile_Chain?
- non ho capito se dobbiamo considerare solo il viaggio del veicolo di trasporto, tutta la catena di approvvigionamento dall'inizio alla fine, o circa come fatto assumendo molte cose sulla parte prima e dopo il viaggio, ossia considerandole come parte integrante del sistema, ma dandole come assunte/ipotizzate per non rendere eccessivamente complessi i diagrammi e il resto dei documenti ---> dalla trascrizione della video lezione, risulta più chiaro che l'ambito del sistema non si limita al solo tragitto fisico sul mezzo di trasporto, ma copre l'intero ciclo di vita logico e operativo della spedizione. Nel modello dei requisiti e nella vista dei concetti/comportamenti di Objectiver, il ciclo di vita comprende diverse fasi sequenziali ben definite:
 - fase preliminare e di configurazione: creazione dell'entità logica della spedizione nel sistema, definizione dei parametri ambientali attesi e delle relative soglie di allarme in base al tipo di bene, registrazione e associazione fisica di uno o più dispositivi di rilevamento al lotto o al contenitore;
 - fase di stoccaggio e preparazione (pre-partenza): monitoraggio delle condizioni nei magazzini o nelle celle frigorifere prima della partenza, verifica dello stato di carica della batteria e applicazione della configurazione iniziale al sensore tramite gateway fisso o mobile;
 - fase di transito e viaggio attivo: avvio della sessione di monitoraggio, acquisizione periodica e memorizzazione locale dei campioni ambientali a bordo sensore, sincronizzazione asincrona tramite il gateway (autista/mezzo), rilevamento e notifica tempestiva di eventuali gradienti anomali o superamenti di soglia, gestione di eventuali stati di pausa/ripresa del viaggio;
 - fase di arrivo e consegna finale: terminazione del monitoraggio al raggiungimento della destinazione, chiusura della raccolta dati da parte del dispositivo, dissociazione del sensore (che torna disponibile nel parco dispositivi per un nuovo utilizzo), generazione del report storico certificato e validazione finale della conformità della merce (accettazione o rifiuto/riserva);

Come conseguenza di quest'ultimo punto, dove allarghiamo la cerchia degli aspetti da monitorare a più parti del 'viaggio' del prodotto, aggiungerei cmq delle assunzioni per evitare di dover spiegare ogni minimo dettaglio nei diagrammi. Per esempio:

1. assumerei che chi crea la spedizione e associa i dispositivi ai prodotti, selezioni la grandezza corretta e imposti le soglie corrette del prodotto;

- gateway di ponte per la connessione internet tra sensori prodotti e cloud: soluzione del prof è usare cell dell'autista o cell del veicolo, altra soluzione potrebbe essere un dispositivo terzo (es. black box) messo dentro il veicolo e dotato di sim (ma circa siamo lì non cambia molto); forse con il cell dell'autista dobbiamo porre delle assunzioni sulla batteria e l'operabilità del cellulare, mentre con la black box dobbiamo capire l'alimentazione e mettere delle assunzioni o ipotesi di dominio anche lì; ---> concordato con Mattia, teniamo lo smartphone autista, spiegando però nel RD che questa non è una scelta a caso ma un Fit Criteria decisionale;
- per l'utente finale direi di tenere il Qrcode che dice il prof, tanto non mi sembra un aspetto così rilevante; ---> risultato della trascrizione della videolezione: l'approccio del QR code è corretto ed essenziale. come ha sottolineato il docente, l'applicazione finale per il consumatore non rientra nei confini del vostro software (*software-to-be*). il vostro sistema deve semplicemente esporre un'interfaccia standard (API) e generare un identificativo univoco (il payload codificato nel QR code) affinché un sistema di terze parti o un portale web esterno possa richiamare i dati storici e validare il certificato di conformità;
- il documento originale (conferma o dimmi se dico stronzate) non dice mai che specifici attori devono poter associare/dissociare i dispositivi. L'esatta mappatura delle responsabilità penso sia una scelta progettuale. Per la dashboard invece mi pare debba essere consultabile da praticamente tutti (cliente, enti di certificazione, manager, autista). Sempre collegato, per fare un esempio, l'autista può mettere in pausa la spedizione se si ferma in autogrill, o solo il manager può farlo? L'autista può associare un sensore a una spedizione o l'associazione è configurabile solo dal cloud prima della partenza? ---> risultato della videolezione: la trascrizione non formalizza una matrice di responsabilità rigida, lasciando questa modellazione come scelta progettuale del gruppo:
 - visibilità della dashboard: la dashboard deve essere accessibile a tutti gli attori, ma con viste e permessi profilati. il manager ha una vista globale su tutte le tratte e la configurazione; l'autista accede solo alla vista operativa del carico sul proprio mezzo e riceve le allerte attive; il cliente/ente certificatore ha accesso in sola lettura allo storico delle sole spedizioni di propria competenza;
 - azione di pausa (es. sosta in autogrill): concettualmente la pausa della spedizione non deve sospendere la registrazione dei dati fisici da parte del sensore (la merce continua a deperire anche a veicolo fermo), bensì indicare uno stato logico del viaggio. è opportuno che l'autista possa segnalare la sosta dal proprio dispositivo per tracciare la deviazione o la sosta programmata, mentre il manager mantiene il controllo su blocchi amministrativi o interruzioni anomale;
 - associazione/dissociazione sensori: la scelta più solida nei diagrammi di responsabilità è assegnare la creazione della spedizione e la definizione delle policy al manager nel cloud, consentendo però all'operatore sul campo (magazziniere o autista tramite app mobile) di eseguire l'associazione fisica

iniziale (es. scansionando il codice del sensore o un barcode) e la dissociazione alla consegna;

- per il progetto, ci sono tanti oggetti fragili da considerare, e in generale l'approccio richiesto dal prof è specificatamente general purpose (gioco di parole :). Quello che non ho capito è se abbia senso o meno utilizzare delle descrizioni generiche dove bisogna tenerle generiche (es. come per gli argomenti trattati in questo documento), e se invece per diagrammi e strutture che necessitano dell'assenza di ambiguità usare dei casi d'uso specifico come, per esempio, i farmaci, impostando quindi delle soglie di temperatura e umidità precise, di tolleranza, ecc ecc. ----> l'approccio migliore per conciliare generalità e rigore metodologico prevede una netta separazione tra modello dei dati e istanze di configurazione: nel modello concettuale e nei diagrammi KAOS (Objectiver): mantenere le entità e i goal a livello astratto e parametrizzato. entità come BeneTracciato, ParametroAmbientale, ProfiloDiTolleranza e SogliaAllarme consentono di descrivere qualsiasi tipo di merce fragile senza vincolare la struttura logica al solo farmaco o alla sola opera d'arte. nella specifica IEEE 830 e nelle descrizioni dei comportamenti: definire il modello generale e poi istanziare scenari applicativi specifici ed esaustivi (es. catena del freddo criogenica per farmaci a -40 °C con tolleranza stretta, oppure trasporto opere d'arte con vincoli su umidità e vibrazioni continue) all'interno delle tabelle dei requisiti, dei casi d'uso o delle post-condizioni, eliminando qualsiasi ambiguità interpretativa tramite valori numerici concreti.

Diagrammi

DETTAGLIO DELLE 6 FASI OPERATIVE (con divisione dei compiti)

FASE 1: Vista Comportamentale Preliminare (Scenari Positivi) ----> MATTIA

- Diagramma da fare: Diagramma di Sequenza / Event Trace (Scenari Positivi).
- Perché si parte da qui: Gli scenari descrivono esempi concreti di interazione nel mondo reale. Poiché sono espressivi e scritti in linguaggio naturale, rappresentano lo strumento ideale per l'elicitazione iniziale con gli stakeholder e per superare la naturale astrattezza dei primi obiettivi.
- Cosa produce per il passo successivo: Analizzando i singoli scambi di messaggi lungo le linee della vita degli attori, ci si pongono domande di tipo "WHY?" (perché l'autista fa questa azione? → sblocca i Goal ad alto livello) e "WHY NOT?" (cosa succede se il sensore si spegne qui? → fa emergere gli Ostacoli).

FASE 2: Vista Intenzionale e dei Rischi (Goal & Obstacle Diagram) ----> MARTIN

- Diagrammi da fare:
 - Goal Diagram (AND/OR Refinement Tree).
 - Obstacle Diagram (Analisi dei Rischi).
- Perché si fa ora: Questa è la colonna vertebrale logica dell'intero progetto (il "WHY"). Spacca gli obiettivi di business fino ad arrivare ai goal foglia: i Requisiti (prescrittivi per la macchina software) e le Aspettative (prescrittive per l'ambiente). Sotto i goal foglia innestate l'analisi dei rischi, trovando gli Ostacoli e i relativi goal di risoluzione (Resolution) che arricchiscono l'albero dei goal con le contromisure di sicurezza.

- Cosa produce per il passo successivo:
 - Ogni concetto, sostantivo o termine menzionato nelle definizioni e specifiche dei Goal deve trovare una sua rappresentazione strutturale formale.
 - La lista dei Goal foglia definisce l'elenco esatto delle responsabilità che andranno assegnate agli agenti.

FASE 3: Vista Strutturale (Class Diagram dei Concetti) ---> MATTIA

- Diagramma da fare: UML Class Diagram Concettuale.
- Perché si fa ora: Avendo definito l'albero dei Goal, avete l'elenco esatto dei termini da mappare. In questa fase si modella esclusivamente il Mondo del Problema (entità, associazioni strutturali, eventi e attributi) escludendo qualsiasi metodo software.
- Regola di coerenza inter-vista sbloccata (Rule 1): Ogni concetto menzionato in un Goal deve esistere come classe, attributo o associazione nel Class Diagram, e viceversa.
- Cosa produce per il passo successivo: Fornisce le variabili di stato (gli attributi delle classi, es. temperatura, statoSpedizione, livelloBatteria) che gli agenti dovranno monitorare o controllare per far funzionare il sistema.

FASE 4: Vista delle Responsabilità (Agent, Context & Dependency) ---> MARTIN

- Diagrammi da fare:
 - Agent Diagram (Responsibility Assignment).
 - Context Diagram (Interfacce Fisiche / Condivisione Variabili).
 - Dependency Diagram (Strategic Dependencies / Vulnerabilità).
- Perché si fa ora: Avendo i goal foglia (Fase 2) e le variabili fisiche da controllare (Fase 3), potete mappare gli Agenti (esagoni).
 - L'Agent Diagram assegna la responsabilità di ciascun goal foglia a un singolo agente (Software-to-be o Ambiente).
 - Mappate le Capabilities dell'agente tramite archi che dicono quali variabili del Class Diagram l'agente può leggere (monitoring) o scrivere (control).
 - Il Context Diagram si ricava automaticamente: se l'agente A controlla la variabile X e l'agente B la monitora per fare il suo lavoro, disegnate un canale di interfaccia fisica da A a B.
 - Il Dependency Diagram evidenzia le milestone: se la parte di sistema fallisce nel metterti a disposizione un dato, il tuo agente non può raggiungere il suo obiettivo.
- Regola di coerenza inter-vista sbloccata (Rule 2): Ogni agente deve essere responsabile di almeno un goal foglia e non possono esserci goal orfani. Una variabile di stato può essere controllata (set) al massimo da un solo agente nell'intero sistema.
- Cosa produce per il passo successivo: Identifica chi è l'esecutore (performer) di ogni singola operazione del sistema.

FASE 5: Vista Funzionale (Operationalization Diagram) ---> ENTRAMBI (oppure Martin Operationalization e Mattia Use Case)

- Diagramma da fare: Operationalization Diagram / Use Case Diagrams.
- Perché si fa ora: Qui traduciamo gli obiettivi statici (Goal) in azioni dinamiche (Operazioni).

- Per ogni goal comportamentale foglia, usiamo l'analisi dei fluenti per identificare l'operazione che "inizia" un determinato stato (lo rende vero) e quella che lo "termina" (lo rende falso), definendo DomPre e DomPost.
 - Rafforziamo le operazioni applicando i vincoli prescrittivi: ReqPre (permessi di sicurezza), ReqTrig (obblighi di innesco) e ReqPost (vincoli di output).
- Regola di coerenza inter-vista sbloccata (Rule 3): Ogni operazione deve essere collegata a un goal foglia tramite un link di operazionalizzazione. Gli input e output dell'operazione devono corrispondere a variabili reali dichiarate nel Class Diagram, e l'esecutore deve coincidere con l'agente responsabile di quel goal.
- Cosa produce per il passo successivo: I parametri delle operazioni (DomPre, ReqPre, ReqTrig) diventano matematicamente le transizioni e le condizioni di guardia della macchina a stati.

FASE 6: Vista Comportamentale (State Machines & Scenari Negativi)

- Diagrammi da fare:
 - Diagrammi di Sequenza con Scenari Negativi ed Eccezionali. ---> Mattia
 - UML State Machine Diagrams. ---> Martin
- Perché si fa ora: È il traguardo finale. Modella lo storico e l'evoluzione dinamica e continua delle classi nel tempo.
 - Costruite una macchina a stati per ogni variabile controllata del Class Diagram.
 - Gli stati semplici corrispondono ai valori degli attributi della classe.
 - Le transizioni sono etichettate con le operazioni definite nella Fase 5, e le condizioni di guardia corrispondono ai loro permessi (ReqPre).
 - Prendete gli Scenari Negativi (quelli che mostrano comportamenti proibiti, es. "la spedizione viene avviata ma il sensore è spento") e usateli per verificare che le guardie della macchina a stati blocchino effettivamente le transizioni pericolose.
- Regola di coerenza inter-vista sbloccata (Rule 4): Ogni transizione della macchina a stati deve essere mappata su un'operazione definita nella vista funzionale, e ogni stato deve essere coerente con i vincoli statici del Class Diagram.

RIASSUNTO DELLE COERENZE (DA TENERE SOTTO MANO)

Nome Diagramma	Cosa leggi nei Nodi	Cosa leggi negli Archi (Link)	Consistenza con altre viste
Goal Diagram	Goal, Sotto-Goal, Requirements, Expectations	AND/OR Refinement, Responsibility, Concern, Obstruction	Ogni concetto → Classe nel Class Diagram
Obstacle Diagram	Goal, Ostacoli (parallelogrammi a sinistra)	AND/OR Refinement, Obstruction, Goal Resolution	Ostacolo = Negazione logica del Goal foglia

Class Diagram	Entità, Associazioni, Agenti, Eventi	Relazioni statiche (Specializzazione, Aggregazione)	No operazioni software, no puntatori
Agent Diagram	Agenti (esagoni), Variabili di stato	Monitoring (lettura), Control (scrittura)	1 Variabile controllata → max 1 Agente
Operationalization	Operazioni (ellissi), Variabili di input/output	Operationalization link (pre-condizioni, trigger, post-condizioni)	Esecutore dell'operazione = Agente responsabile del goal
State Machine	Stati logici delle classi, Transizioni	Eventi, Guardie, Azioni	Transizione = Operazione; Stato = Valore attributo Classe

Regole e trabocchetti diagrammi

Vista 1: Il Diagramma dei Goal (Vista Intenzionale)

Mostra il "perché" del sistema, raffinando i grandi obiettivi di business fino ai requisiti e alle aspettative foglia.

- La Sintassi Visiva:
 1. Goal / Sotto-Goal: Parallelogramma azzurro inclinato a destra.
 2. Requirement: Parallelogramma con bordo spesso (assegnato esclusivamente ad agenti del Software-to-be).
 3. Expectation: Parallelogramma giallo (aspettativa prescrittiva assegnata ad agenti dell'ambiente, es. l'autista).
 4. Raffinamento AND: Pallino giallo di congiunzione (tutti i sotto-goal devono essere veri per soddisfare il padre).
 5. Raffinamento OR: Frecce separate con pallini indipendenti (alternative di design o automazione).
- Trabocchetti & Bad Smells:
 1. Confondere Goal e Operazioni: I Goal devono essere espressi come stati desiderati usando il participio passato (es. Maintain [DataDigitallySigned]), mentre le operazioni usano l'infinito (es. SignData).
 2. Sotto-goal AND incompleti o ridondanti: Le condizioni in AND devono essere contemporaneamente necessarie e sufficienti per implicare logicamente il goal padre (SubG,Exp,Dom=G). Se puoi rimuovere un sotto-goal senza far saltare il padre, il diagramma è ridondante.
- L'Analisi degli Ostacoli (Risk Analysis): Sotto i goal foglia, dovete innestare la gestione dei rischi. L'ostacolo (parallelogramma arancione inclinato a sinistra) è la

negazione logica del goal foglia. Va raffinato per trovarne le cause fisiche e risolto introducendo un nuovo goal di contromisura collegato tramite un Goal Resolution link.

Vista 2: Il Class Diagram (Vista Strutturale Concettuale)

Definisce le entità statiche del mondo reale su cui opera il sistema.

- La Sintassi Visiva: Usa la notazione delle classi UML, ma con una distinzione vitale: descrive solo il Mondo del Problema, non il codice!
 1. Entità: Oggetti passivi e indipendenti (es. DispositivoIoT, Spedizione).
 2. Associazioni: Relazioni strutturali dipendenti (es. CensitoIn, AssegnatoA).
 3. Agenti: Ruoli attivi in grado di monitorare/controllare variabili (es. Autista, Manager).
 4. Eventi: Cose istantanee che esistono in un solo stato (es. InizioSpedizione).
- Trabocchetti & Bad Smells:
 1. Niente metodi o operazioni: Le classi non devono contenere operazioni (es. niente funzioni come creaSpedizione()), poiché il design del software è escluso dall'Ingegneria dei Requisiti.
 2. Evitare i puntatori come attributi: Non inserire chiavi esterne (es. scrivere l'attributo id_sensore dentro la classe Spedizione). Le relazioni devono essere rappresentate esclusivamente tramite linee di associazione bidirezionali esplicite.
 3. No a relazioni dinamiche o di controllo: Non inserire archi che indicano azioni (es. associazioni chiamate "Invia", "Monitora" o "Aggiorna"). Il Class Diagram deve mostrare solo relazioni statico-strutturali.

Vista 3: L'Agent Diagram (Vista delle Responsabilità)

Assegna formalmente la responsabilità dei goal foglia del primo diagramma ai singoli agenti (Software o Ambiente).

- La Sintassi Visiva:
 - Agenti: Rappresentati come esagoni.
 - Capabilities: Freccie che collegano gli agenti alle variabili di stato del Class Diagram, marcate come monitoring (lettura / get) o control (scrittura / set).
- Trabocchetti & Bad Smells:
 - Regola del controllore unico: Una variabile di stato del Class Diagram può essere controllata (set) al massimo da un solo agente nell'intero sistema per evitare conflitti concorrenti.
 - Violazione della realizzabilità: Non puoi assegnare la responsabilità di un goal a un agente che non ha le capacità fisiche per monitorare o controllare le variabili coinvolte in quel goal.
- Sotto-diagrammi richiesti:
 - Context Diagram: Mostra le interfacce tra agenti tramite frecce etichettate con le variabili condivise (se l'Agente A controlla X e l'Agente B lo monitora, si disegna una freccia da A a B).
 - Dependency Diagram: Mostra come il fallimento di un agente dependee su un sotto-obiettivo possa causare il fallimento a catena di un agente depender.

Vista 4: L'Operationalization Diagram (Vista Funzionale)

Definisce le transizioni di stato (operazioni software o procedure manuali dell'ambiente) necessarie a soddisfare i goal foglia.

- La Sintassi Visiva:
 - Operazione: Ellisse etichettata con un verbo all'infinito (es. AvviaSpedizione).

- È collegata verso l'alto tramite un link di operazionalizzazione ai goal foglia che soddisfa, e verso i lati ai dati di input e output presi dal Class Diagram.
- Le Condizioni Logiche (Rigorosissime): Ogni operazione deve definire cinque tipi di vincoli testuali:
 - DomPre (Pre-condizione di dominio): Cosa deve essere fisicamente vero nel mondo affinché l'operazione possa esistere (es. la spedizione è in stato "Creata").
 - DomPost (Post-condizione di dominio): L'effetto fisico immediato dell'operazione (es. la spedizione è in stato "Avviata").
 - ReqPre (Pre-condizione richiesta): Un permesso di sicurezza (es. l'operazione può avvenire solo se l'utente è un Manager loggato).
 - ReqTrig (Condizione di innesco): Un obbligo (l'operazione deve avvenire non appena questa condizione diventa vera).
 - ReqPost (Post-condizione richiesta): Vincoli aggiuntivi sull'output per soddisfare i requisiti di qualità o non funzionali.
- Trabocchetti & Bad Smells: L'agente che è stato dichiarato responsabile del goal foglia nella Vista 3 deve essere rigorosamente l'unico esecutore (performer) delle operazioni che operazionalizzano quel goal.

Vista 5: Il Behavioral Diagram (Vista Comportamentale)

Modella come evolve dinamicamente il sistema nel tempo.

- Diagramma di Sequenza (Event Trace): Mostra una timeline verticale per ogni istanza di agente e i messaggi scambiati (che corrispondono alle operazioni). Ricordati di includere scenari positivi (flusso normale) e scenari negativi (utilizzando i blocchi neg per mostrare interazioni proibite che violano i goal di sicurezza).
- Macchina a Stati (State Machine): Definisce gli stati logici delle classi. Deve esserci una macchina a stati per ogni variabile controllata da un agente. Le transizioni tra gli stati sono etichettate con il nome delle operazioni, mentre le condizioni di ReqPre (permessi) si trasformano graficamente nelle "guardie" (es. [batteria > 15%]).